Refer below links for video tutorials:

1. https://youtu.be/lG45bWUxBSU?si=aGgGbe0IufFbHADc (Tamil)
2. https://youtu.be/poo0BXryffI?si=GDmuY_4xtjk3OCiv (Tamil)
3. https://youtube.com/playlist?list=PL9ooVrP1hQOGoYVit61V7Y1uPzfnBvN86&si=pkfbZksGlXh1Jyye (English)
4. https://youtu.be/Zi-Q0t4gMC8?si=92tJLXf1gS7izAhs (English)

JAVA SCRIPT

Day 14: Introduction to JavaScript

Topics

1. **What is JavaScript?**

- **Definition**: JavaScript is a versatile, high-level programming language primarily used to create interactive and dynamic content on web pages. It can manipulate HTML and CSS, handle events, validate forms, and more, making it an essential tool in modern web development.
- **Key Characteristics**:
  - **Client-Side**: Runs directly in the web browser, enabling dynamic content without needing to reload the page.
  - **Event-Driven**: Responds to user actions like clicks, form submissions, or keystrokes.
  - **Interpreted Language**: Executed line by line in real-time by the browser's JavaScript engine.
- **Example Usage**:
  - **Form Validation**: Ensuring user input meets specific criteria before submission.
  - **Interactive Maps**: Updating map content without refreshing the page.
  - **Dynamic Content**: Displaying or hiding elements, changing styles, or updating text in response to user actions.

2. **Basic Syntax and Structure**

- **Structure**:
  - **Statements**: Instructions written in JavaScript, typically ending with a semicolon (;).
  - **Variables**: Used to store data values, declared using var, let, or const.
  - **Functions**: Reusable blocks of code that perform a specific task.
  - **Operators**: Symbols used for calculations (+, -, *, /) or comparisons (==, ===, >, <).
- **Example**:

```
let name = "John"; // Variable declaration
let age = 25;
let isStudent = true; // Boolean value

function greet() {
    console.log("Hello, " + name); // Function that prints a greeting
}

greet(); // Calling the function
```

  - **Explanation**: This script declares variables and a function, then calls the function to print a greeting in the console.

3. **Comments and Console**

- **Comments**:
  - **Single-Line Comment**: Written with // and used for brief explanations or to disable code.

```
// This is a single-line comment
let x = 10; // Declare a variable
```

- **Multi-Line Comment**: Written with /* */ and used for longer explanations or block commenting.

```
/*
This is a multi-line comment
It spans multiple lines
*/
let y = 20;
```

- **Console**:
  - **console.log()**: A method used to print output or debug information in the browser's console.
  - **Example**:

```
console.log("Hello, World!"); // Prints "Hello, World!" in the console
let sum = 5 + 3;
console.log("The sum is: " + sum); // Prints "The sum is: 8"
```

  - **Explanation**: The console.log() method is crucial for debugging and checking the output of your JavaScript code.

Tasks

1. **Task 1: Set Up Your Development Environment**
   - **Description**: Install a text editor like Visual Studio Code, create a simple HTML file, and link an external JavaScript file. Use the browser's developer tools to view the output.
   - **Objective**: Familiarize yourself with the tools needed for JavaScript development.
2. **Task 2: Practice Writing Basic JavaScript**
   - **Description**: Write JavaScript code to declare variables, create a simple function, and use console.log() to display messages and variable values in the console.
   - **Objective**: Understand the basic syntax, structure, and debugging techniques in JavaScript.

Day 15: Variables and Data Types

Topics

1. **Understanding Variables (var, let, const)**

   - **Definition**: Variables are containers used to store data that can be referenced and manipulated in a program. In JavaScript, variables are declared using the keywords var, let, or const.

   - **Types of Variable Declarations**:

     1. **var**:

        a. **Scope**: Function-scoped, meaning it is accessible throughout the function where it is declared.

        b. **Hoisting**: var declarations are hoisted to the top of their scope, but not the assignments.

        c. **Example**:

```
var x = 10;
function example() {
    var y = 20;
    console.log(x + y); // 30
}
example();
console.log(x); // 10
// console.log(y); // Error: y is not defined
```

     2. **let**:

a) **Scope**: Block-scoped, meaning it is only accessible within the block ({}) where it is declared.

b) **Hoisting**: let is also hoisted, but it is not initialized, so using it before declaration results in a ReferenceError.

c) **Example**:

```
let z = 5;
if (true) {
    let z = 10;
    console.log(z); // 10 (local scope)
}
console.log(z); // 5 (global scope)
```

3. **const**:

a) **Scope**: Block-scoped, like let.

b) **Immutability**: Values assigned to const variables cannot be reassigned, making it useful for constants.

c) **Example**:

```
const PI = 3.14;
console.log(PI); // 3.14
// PI = 3.15; // Error: Assignment to constant variable
```

2. JavaScript Data Types

- **Definition**: JavaScript provides various data types to handle different kinds of data. Understanding these data types is fundamental for working with JavaScript.

- **Primary Data Types**:

  1. **string**: Represents textual data enclosed in quotes.

  - **Example**:

```
let name = "Alice";
console.log(name); // Alice
```

2. **number**: Represents numeric data, including integers and floating-point numbers.

- **Example**:

```
let age = 30;
let price = 99.99;
console.log(age); // 30
console.log(price); // 99.99
```

3. **boolean**: Represents true or false values, often used in conditional statements.

- **Example**:

```
let isStudent = true;
let hasGraduated = false;
console.log(isStudent); // true
```

4. **undefined**: Represents a variable that has been declared but not assigned a value.

- **Example**:

```
let x;
console.log(x); // undefined
```

5. **null**: Represents the intentional absence of any object value, often used to reset or clear a variable.

- **Example**:

```
let person = null;
console.log(person); // null
```

6. **symbol**: Represents a unique identifier, often used in object properties to avoid conflicts.

- **Example**:

```
let sym = Symbol("unique");
console.log(sym); // Symbol(unique)
```

7. **object**: Represents a collection of key-value pairs, used to store complex data structures.

- **Example**:

```
let car = {
    make: "Toyota",
    model: "Corolla",
    year: 2020
};
console.log(car.make); // Toyota
```

Tasks

1. **Task 1: Practice Declaring Variables**
   - **Description**: Write a JavaScript script where you declare variables using var, let, and const. Experiment with their scopes by declaring them inside and outside of functions or blocks.
   - **Objective**: Understand the differences between var, let, and const and how variable scope affects your code.
2. **Task 2: Work with Different Data Types**
   - **Description**: Create a JavaScript program that demonstrates the use of all the primary data types. Store values in variables, then print them using console.log() to understand how each type is used and displayed.
   - **Objective**: Familiarize yourself with JavaScript's basic data types and how to use them effectively in your code.

Day 16: Operators and Expressions

Topics

1. **Arithmetic Operators (+, -, *, /, %)**
   - **Definition**: Arithmetic operators are used to perform basic mathematical operations on numbers.
   - **Addition (+)**:
     - **Definition**: Adds two operands.
     - **Example**:

```
let a = 5;
let b = 3;
let sum = a + b; // 8
```

     - **Explanation**: Here, 5 + 3 equals 8, so sum is assigned the value 8.
   - **Subtraction (-)**:
     - **Definition**: Subtracts the right operand from the left operand.
     - **Example**:

```
let difference = a - b; // 2
```

     - **Explanation**: Subtracting 3 from 5 results in 2.
   - **Multiplication (*)**:
     - **Definition**: Multiplies two operands.
     - **Example**:

```
let product = a * b; // 15
```

- **Explanation**: Multiplying 5 and 3 results in 15.

- **Division (/)**:
  - **Definition**: Divides the left operand by the right operand.
  - **Example**:

```
let quotient = a / b; // 1.6667
```

  - **Explanation**: Dividing 5 by 3 results in 1.6667.
- **Modulus (%)**: Returns the remainder of a division.
  - **Definition**: Returns the remainder of the division of two operands.
  - **Example**:

```
let remainder = a % b; // 2
```

  - **Explanation**: The remainder of 5 / 3 is 2.

2. **Assignment Operators (=, +=, -=, etc.)**
   - **Definition**: Assignment operators are used to assign values to variables.
   - **Examples**:
   - **Simple Assignment (=)**:
     - **Definition**: Assigns the value of the right operand to the left operand.
     - **Example**:

```
let x = 10;
```

     - **Explanation**: x is assigned the value 10.
   - **Addition Assignment (+=)**: Adds the right operand to the left operand and assigns the result to the left operand.

- **Definition**: Adds the right operand to the left operand and assigns the result to the left operand.
- **Example**:

```
x += 5; // x = x + 5; x is now 15
```

- **Explanation**: x is updated by adding 5 to its current value.

- **Subtraction Assignment (-=)**:
  - **Definition**: Subtracts the right operand from the left operand and assigns the result to the left operand.
  - **Example**:

```
x -= 3; // x = x - 3; x is now 12
```

  - **Explanation**: 3 is subtracted from x, updating its value to 12.

- **Multiplication Assignment (*=)**:
  - **Definition**: Multiplies the left operand by the right operand and assigns the result to the left operand.
  - **Example**:

```
x *= 2; // x = x * 2; x is now 24
```

  - **Explanation**: x is multiplied by 2, resulting in 24.

- **Division Assignment (/=)**:
  - **Definition**: Divides the left operand by the right operand and assigns the result to the left operand.
  - **Example**:

```
x /= 4; // x = x / 4; x is now 6
```

  - **Explanation**: x is divided by 4, resulting in 6.

3. **Comparison Operators (==, ===, !=, !==, >, <, >=, <=)**

- **Definition**: Comparison operators are used to compare two values and return a boolean (true or false).
- **Examples**:
- **Equality (==)**:
  - **Definition**: Checks if two values are equal, performing type conversion if necessary.
  - **Example**:

```
console.log(5 == '5'); // true
```

  - **Explanation**: JavaScript converts the string '5' to a number, so 5 == '5' evaluates to true.
- **Strict Equality (===)**:
  - **Definition:** Checks if two values are equal and of the same type.
  - **Example:**

```
console.log(5 === '5'); // false
```

  - **Explanation:** Since 5 is a number and '5' is a string, they are not strictly equal.
- **Inequality (!=)**:
  - **Definition**: Checks if two values are not equal, performing type conversion if necessary.
  - **Example**:

```
console.log(5 != '5'); // false
```

  - **Explanation**: Again, JavaScript converts '5' to a number, so 5 != '5' evaluates to false.
- **Strict Inequality (!==)**:
  - **Definition**: Checks if two values are not equal or not of the same type.

- **Example**:

```
console.log(5 !== '5'); // true
```

- **Explanation**: 5 and '5' are not of the same type, so this returns true

- **Greater Than (>)**:
  - **Definition**: Checks if the left operand is greater than the right operand.
  - **Example**:

```
console.log(10 > 5); // true
```

  - **Explanation**: 10 is greater than 5, so this returns true.

- **Less Than (<)**:
  - **Definition**: Checks if the left operand is less than the right operand.
  - **Example**:

```
console.log(10 < 5); // false
```

  - **Explanation**: 10 is not less than 5, so this returns false.

- **Greater Than or Equal (>=)**:
  - **Definition**: Checks if the left operand is greater than or equal to the right operand.
  - **Example**:

```
console.log(10 >= 5); // true
```

  - **Explanation**: 10 is greater than or equal to 5, so this returns true.

- **Less Than or Equal (<=)**:
  - **Definition**: Checks if the left operand is less than or equal to the right operand.
  - **Example**:

```
console.log(10 <= 5); // false
```

- **Explanation**: 10 is not less than or equal to 5, so this returns false.

4. **Logical Operators (&&, ||, !)**

- **Definition**: Logical operators are used to combine or invert boolean values.
- **Examples**:
- **Logical AND (&&)**: Returns true if both operands are true.
  - **Definition**: Returns true if both operands are true.
  - **Example**:

```
let a = true, b = false;
console.log(a && b); // false
```

  - **Explanation**: Since b is false, a && b returns false.
- **Logical OR (||)**: Returns true if at least one operand is true.
  - **Definition**: Returns true if at least one operand is true.
  - **Example**:

```
console.log(a || b); // true
```

  - **Explanation**: Since a is true, a || b returns true.
- **Logical NOT (!)**: Inverts the boolean value of an operand.
  - **Definition**: Inverts the boolean value of the operand.
  - **Example**:

```
console.log(!a); // false
```

  - **Explanation**: Since a is true, !a returns false.

Tasks

1. **Task 1: Practice Arithmetic and Assignment Operators**
    - **Description**: Write a JavaScript script where you perform various arithmetic operations on numbers. Use assignment operators to update variables based on the results.
    - **Objective**: Understand how arithmetic and assignment operators work together in JavaScript.
2. **Task 2: Explore Comparison and Logical Operators**
    - **Description**: Create a program that compares different values using comparison operators. Combine multiple conditions using logical operators to produce complex boolean expressions.
    - **Objective**: Learn how to use comparison and logical operators to evaluate conditions in JavaScript.

Day 17: Control Flow in JavaScript

1. If, Else If, Else Statements

These statements allow you to execute different blocks of code based on conditions.

- **If Statement**:
  - **Definition**: Executes a block of code if a specified condition is true.
  - **Syntax**:

```
if (condition) {
    // code to be executed if the condition is true
}
```

  - **Example**:

```
let age = 18;
if (age >= 18) {
    console.log("You are an adult.");
}
```

  - **Explanation**: If age is 18 or more, the message "You are an adult." is printed to the console.
- **Else If Statement**:
  - **Definition**: Specifies a new condition to test if the first condition is false.
  - **Syntax**:

```
if (condition1) {
    // code to be executed if condition1 is true
} else if (condition2) {
    // code to be executed if condition2 is true
}
```

- **Example**:

```
let score = 75;
if (score >= 90) {
    console.log("Grade A");
} else if (score >= 80) {
    console.log("Grade B");
} else if (score >= 70) {
    console.log("Grade C");
}
```

- **Explanation**: Depending on the score, the code outputs the corresponding grade.

- **Else Statement**:
  - **Definition**: Executes a block of code if none of the previous conditions are true.
  - **Syntax**:

```
if (condition1) {
    // code to be executed if condition1 is true
} else if (condition2) {
    // code to be executed if condition2 is true
} else {
    // code to be executed if all conditions are false
}
```

  - **Example**:

```
let score = 65;
if (score >= 90) {
    console.log("Grade A");
} else if (score >= 80) {
    console.log("Grade B");
} else if (score >= 70) {
    console.log("Grade C");
} else {
    console.log("Fail");
}
```

- **Explanation**: If score is below 70, the message "Fail" is printed.

---

2. Switch Statements

The switch statement is used to perform different actions based on different conditions. It is an alternative to using multiple if...else statements.

- **Syntax**:

```
switch(expression) {
    case value1:
        // code to be executed if expression equals value1
        break;
    case value2:
        // code to be executed if expression equals value2
        break;
    // more cases
    default:
        // code to be executed if expression doesn't match any case
}
```

- **Example**:

```
let day = 3;
switch(day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    case 3:
        console.log("Wednesday");
        break;
    case 4:
        console.log("Thursday");
        break;
    case 5:
        console.log("Friday");
        break;
    default:
        console.log("Weekend");
}
```

- **Explanation**: If day equals 3, the output will be "Wednesday". If no case matches, the default code is executed.

Tasks for Day 17

1. **Task 1: Use If-Else Statements**
   - **Description**: Write a program that checks the weather and outputs different messages based on whether it's sunny, rainy, or cloudy.
   - **Objective**: Practice using if, else if, and else statements to control the flow of a program.
2. **Task 2: Implement a Switch Statement**
   - **Description**: Create a program that outputs the name of a day of the week based on a given number (1-7). Use a switch statement to handle each case.
   - **Objective**: Understand how to use switch statements to simplify multiple conditions.

Day 18: Functions in JavaScript

1. Function Declaration and Invocation

- **Function Declaration**:
  - **Definition**: A function is a reusable block of code that performs a specific task. You can define a function using the function keyword.
  - **Syntax**:

```
function functionName() {
    // code to be executed
}
```

  - **Example**:

```
function greet() {
    console.log("Hello, World!");
}
```

  - **Explanation**: The greet function is declared, and it contains a single line of code that prints "Hello, World!" to the console.
- **Function Invocation**:
  - **Definition**: To execute a function, you need to "invoke" or "call" it by using its name followed by parentheses.
  - **Syntax**:

```
functionName();
```

  - **Example**:

```
greet(); // Output: Hello, World!
```

- **Explanation**: The greet function is called, and the message is printed to the console.

---

2. Function Parameters and Return Values

- **Function Parameters**:
  - **Definition**: Parameters are variables that are passed into a function to provide it with input data.
  - **Syntax**:

```
function functionName(parameter1, parameter2) {
    // code to be executed using parameters
}
```

  - **Example**:

```
function greetUser(name) {
    console.log("Hello, " + name + "!");
}
greetUser("Alice"); // Output: Hello, Alice!
```

  - **Explanation**: The greetUser function takes a parameter name, and when the function is called with "Alice", it prints "Hello, Alice!" to the console.

- **Return Values**:
  - **Definition**: A function can return a value to the code that called it using the return keyword.
  - **Syntax**:

```
function functionName() {
    return value;
}
```

- **Example**:

```
function add(a, b) {
    return a + b;
}
let sum = add(3, 4); // sum = 7
console.log(sum); // Output: 7
```

- **Explanation**: The add function returns the sum of a and b. When called with 3 and 4, it returns 7, which is then stored in the sum variable.

---

3. Arrow Functions (=>)

- **Definition**: Arrow functions provide a more concise syntax for writing functions and also bind this lexically, which means they do not create their own this context.

- **Syntax**:

```
const functionName = (parameter1, parameter2) => {
    // code to be executed
};
```

- **Example**:

```
const multiply = (a, b) => a * b;
console.log(multiply(3, 4)); // Output: 12
```

- **Explanation**: The multiply arrow function takes two parameters and returns their product. When called with 3 and 4, it returns 12.

Tasks for Day 18

1. **Task 1: Create and Invoke Functions**
   - **Description**: Write a function that takes a person's name and age as parameters and returns a greeting message that includes their name and age. Call the function with different values and print the results.
   - **Objective**: Practice function declaration, parameters, and return values.
2. **Task 2: Use Arrow Functions**
   - **Description**: Convert a traditional function that calculates the square of a number into an arrow function. Test it with different numbers.
   - **Objective**: Get familiar with arrow function syntax and practice using it to simplify your code.

Day 19: Loops and Iteration in JavaScript

1. for, while, and do...while Loops

Loops allow you to repeat a block of code multiple times, which is particularly useful when working with arrays, lists, or performing repetitive tasks.

- **for Loop**:
  - **Definition**: The for loop is used when the number of iterations is known beforehand.
  - **Syntax**:

```
for (initialization; condition; increment) {
    // code to be executed
}
```

  - **Example**:

```
for (let i = 0; i < 5; i++) {
    console.log("Iteration number: " + i);
}
```

  - **Explanation**: The loop starts with i = 0 and continues as long as i < 5, incrementing i by 1 after each iteration.
- **while Loop**:
  - **Definition**: The while loop is used when the number of iterations is not known and depends on a condition being true.
  - **Syntax**:

```
while (condition) {
    // code to be executed
}
```

- **Example**:

```
let count = 0;
while (count < 5) {
    console.log("Count is: " + count);
    count++;
}
```

- **Explanation**: The loop continues as long as count is less than 5, incrementing count by 1 in each iteration.

- **do...while Loop**:
  - **Definition**: The do...while loop is similar to the while loop, but it guarantees that the loop body is executed at least once, even if the condition is false.
  - **Syntax**:

```
do {
    // code to be executed
} while (condition);
```

  - **Example**:

```
let num = 0;
do {
    console.log("Number is: " + num);
    num++;
} while (num < 5);
```

  - **Explanation**: The loop runs at least once, and then continues as long as num is less than 5.

2. Loop Control with break and continue

These control statements are used to change the flow of loops.

- **break**:
  - **Definition**: The break statement is used to exit a loop immediately, regardless of the loop's condition.
  - **Example**:

```
for (let i = 0; i < 10; i++) {
    if (i === 5) {
        break; // Exit the loop when i equals 5
    }
    console.log(i);
}
```

  - **Explanation**: The loop stops when i equals 5, so only 0 to 4 are printed.
- **continue**:
  - **Definition**: The continue statement skips the rest of the current loop iteration and moves on to the next iteration.
  - **Example**:

```
for (let i = 0; i < 10; i++) {
    if (i % 2 === 0) {
        continue; // Skip the current iteration if i is even
    }
    console.log(i);
}
```

  - **Explanation**: The loop skips printing even numbers, so only odd numbers from 1 to 9 are printed.

Tasks for Day 19

1. **Task 1: Implement a for Loop**
   - **Description**: Write a for loop that iterates from 1 to 10 and prints whether each number is even or odd.
   - **Objective**: Practice using for loops and conditionals within a loop.
2. **Task 2: Use while and do...while Loops**
   - **Description**: Create a while loop that counts down from 10 to 1, and a do...while loop that counts up from 1 to 5.
   - **Objective**: Get comfortable with both while and do...while loops and understand their differences.
3. **Task 3: Apply break and continue**
   - **Description**: Write a for loop that iterates from 1 to 20. Use continue to skip multiples of 3 and break the loop if a number greater than 15 is encountered.
   - **Objective**: Understand how to control loop execution using break and continue statements.

Day 20: Objects and Object Methods in JavaScript

1. Introduction to Objects

- **Definition**: In JavaScript, an object is a collection of related data and functionality, organized as properties (key-value pairs) and methods (functions).
- **Syntax**:

```
const person = {
    firstName: "John",
    lastName: "Doe",
    age: 30,
    greet: function() {
        return "Hello, " + this.firstName;
    }
};
```

- **Explanation**: Here, person is an object with properties (firstName, lastName, age) and a method (greet). The properties store values related to the object, while the method defines behavior associated with the object.

2. Accessing and Modifying Object Properties

- **Accessing Properties**:
  - **Dot Notation**:

```
console.log(person.firstName); // Outputs: John
```

  - **Bracket Notation**:

```
console.log(person['lastName']); // Outputs: Doe
```

- **Explanation**: You can access object properties using dot notation (object.property) or bracket notation (object['property']). Bracket notation is useful when the property name is dynamic or not a valid JavaScript identifier.

- **Modifying Properties**:

```
person.age = 31; // Modifies the age property
person['firstName'] = "Jane"; // Modifies the firstName property
```

  - **Explanation**: Properties of an object can be modified or added using both dot and bracket notation.

---

3. Object Methods and this Keyword

- **Object Methods**:
  - **Definition**: A method is a function that is a property of an object.
  - **Example**:

```
const car = {
    brand: "Toyota",
    model: "Camry",
    start: function() {
        return "The " + this.brand + " " + this.model + " is starting.";
    }
};
console.log(car.start()); // Outputs: The Toyota Camry is starting.
```

  - **Explanation**: The start function is a method of the car object. It uses the this keyword to refer to the current object.

- **this Keyword**:
  - **Definition**: In the context of an object method, this refers to the object on which the method was called.

- **Example**:

```
const user = {
    name: "Alice",
    logName: function() {
        console.log(this.name);
    }
};
user.logName(); // Outputs: Alice
```

- **Explanation**: The this keyword inside logName refers to the user object, allowing the method to access the name property.

Tasks for Day 8

1. **Task 1: Create and Access Object Properties**
   - **Description**: Create an object that represents a book with properties like title, author, and pages. Access and print each property using both dot and bracket notation.
   - **Objective**: Practice creating objects and accessing their properties using different notations.
2. **Task 2: Add and Modify Object Methods**
   - **Description**: Extend the book object by adding a method summary that returns a string summarizing the book. Use the this keyword within the method to access the object's properties.
   - **Objective**: Learn to add methods to objects and understand the use of the this keyword.

Day 21: Events and Basic DOM Manipulation in JavaScript

1. Introduction to the Document Object Model (DOM)

- **Definition**: The DOM is a programming interface for web documents. It represents the structure of a document as a tree of nodes, where each node corresponds to a part of the document (e.g., an element, attribute, or text).
- **Example**: Consider the following HTML structure:

```html
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Welcome to My Page</h1>
    <button id="myButton">Click Me</button>
  </body>
</html>
```

  - In the DOM, this structure is represented as a tree, with the <html> element as the root node, and the <head>, <body>, and other elements as child nodes.

---

2. Event Handling

- **Definition**: Events are actions or occurrences that happen in the browser (e.g., clicking a button, hovering over an element). JavaScript can detect these events and respond to them with functions called event handlers.
- **Common Events**:
  - **click**: Triggered when an element is clicked.
  - **mouseover**: Triggered when the mouse hovers over an element.
  - **keydown**: Triggered when a key is pressed.

- **Example**:

```
const button = document.getElementById('myButton');
button.addEventListener('click', function() {
    alert('Button was clicked!');
});
```

  - **Explanation**: Here, addEventListener is used to attach a click event to the button. When the button is clicked, an alert box with the message "Button was clicked!" is displayed.

---

3. Selecting and Modifying DOM Elements

- **Selecting Elements**:
  - **getElementById**: Selects an element by its ID.

```
const title = document.getElementById('myTitle');
```

  - **querySelector**: Selects the first element that matches a CSS selector.

```
const button = document.querySelector('button');
```

- **Modifying Elements**:
  - **Changing Text Content**:

```
const title = document.getElementById('myTitle');
title.textContent = 'New Title';
```

  - **Changing Styles**:

```
const button = document.querySelector('button');
button.style.backgroundColor = 'blue';
```

- **Explanation**: The DOM provides methods to select elements and modify their properties, such as text content and CSS styles. These modifications are immediately reflected in the browser.

Tasks for Day 21

1. **Task 1: Handle Click Events**

   - **Description**: Create a webpage with a button. When the button is clicked, change the text of a heading element and log a message to the console.
   - **Objective**: Practice handling click events and modifying DOM elements in response.

2. **Task 2: Modify DOM Elements with JavaScript**

   - **Description**: Select a paragraph element and change its text content and background color using JavaScript. Add a hover effect that changes the text color.
   - **Objective**: Learn to select and modify DOM elements, including changing styles and responding to hover events.

Day 22: Comprehensive JavaScript Task

Task 1: Build an Interactive To-Do List

Description: Create a simple to-do list application where users can add tasks, mark them as completed, and remove them from the list. The application should incorporate the following JavaScript concepts:

- **Variables and Data Types**: Use variables to store task information.
- **Functions**: Write functions to add, remove, and mark tasks as completed.
- **Events**: Handle click events to interact with the to-do list (e.g., adding tasks, checking off tasks, and deleting tasks).
- **DOM Manipulation**: Dynamically create and modify DOM elements to display the tasks.
- **Control Flow**: Use if statements to check if a task is completed and apply a different style.

Objective: This task will help you practice creating a fully interactive web application using fundamental JavaScript concepts like functions, event handling, and DOM manipulation.

Task 2: Create a Simple Quiz Application

Description: Develop a basic quiz application that displays a series of questions with multiple-choice answers. When the user selects an answer and submits it, the application should:

- **Functions**: Use functions to evaluate the selected answers and provide feedback.
- **Loops**: Use a for loop to iterate through the quiz questions and display them one by one.
- **Operators**: Use comparison operators to check if the selected answer is correct.

- **Objects and Arrays**: Store questions, choices, and correct answers in an array of objects.
- **DOM Manipulation**: Display the questions and handle user input, updating the DOM based on the user's selections.

Objective: This task will help you apply various JavaScript concepts like objects, arrays, loops, and event handling in the context of a simple application, reinforcing your understanding of core JavaScript principles.